A dialog-based recommendation system integrating machine learning, finite state modelling and rule-based reasoning.

Group: C6

Lenny, L.G., Pam
7128584, Artificial Intelligence, Utrecht University, l.g.pam@students.uu.nl

S. Sedhom
4024987, Artificial Intelligence, Utrecht University, s.sedhom@students.uu.nl

Dirk-Jan, D.J, Schoppema
7170815, Artificial Intelligence, Utrecht University, d.j.schoppema@students.uu.nl

Bram, A.W., Tardijn
9055800, Artificial Intelligence, Utrecht University, a.w.tardijn@students.uu.nl

Abstract

The goal of this project is to design and implement a dialogue-based restaurant recommendation system that enables users to find dining options through text-based interaction fitting their preferences. The system integrates three core components: machine learning, finite state modelling and manually defined reasoning rules. The machine learning module classifies 25,501 user utterances into dialogue acts using four models—logistic regression, support vector machine, multinomial naïve Bayes, and decision tree classifiers. A major limitation of the project is the relatively small and highly duplicated dataset, with only 5,110 unique utterances. Before the machine learning classifiers were trained, the data set was preprocessed, since there were multiple unusable utterances and there were only 5,110 unique utterances. Duplication of data can play a key factor in machine learning. The best-performing classifiers, logistic regression and support vector machine, achieved 98% accuracy. The finite state machine guides the user through the conversation, including greeting, preference collection, suggestion, and closure. The reasoning component applies a fixed set of inference rules to justify or refine recommendations. Combined, these components form a complete conversational system. Future improvements could include enhanced contextual memory, extended reasoning capabilities, and integration with real-world restaurant data sources.

1. Introduction:

This project presents the design and implementation of a dialogue-based restaurant recommendation system that combines rule-based mechanisms with data-driven components to manage user interactions. This generative system helps users to search for suitable restaurants through conversational dialogue. The system asks for preferences like food preference, price range and location preference.

The system consists of three core components. The first is a machine learning (ML) module for classifying user utterances into dialogue acts. The classifiers were trained on a data collection (see Section 2) that underwent extensive preprocessing. The dataset contained 25,501 utterances, of which 23,889 were used for training after cleaning. Due to a high degree of duplication—only 5,110 unique utterances—the dataset posed a significant limitation, as duplication can bias learning outcomes.

The ML module (Section 3) includes four different classifiers—Logistic Regression, Support Vector Machine, Multinomial Naïve Bayes, and Decision Tree—and two baselines. The best models achieved an accuracy of 98%.

The second core component is the finite state machine (FSM) (4). The FSM serves as the central control mechanism for the dialog flow. It interacts with both the ML model and the restaurant manager to guide users through multiple conversation states. The third component is a reasoning module (Section 5) that applies a fixed set of inference rules to draw conclusions about restaurant attributes.

User interaction is entirely text-based, with short and informal messages that mimic natural conversation. The finite state machine (4) has multiple states and transitions which represent a dialog between two agents. The FSM ensures coherent dialog flow, from greeting and preference gathering to restaurant suggestion and conversation closure. Each transition reflects logical system behavior, resulting in a modular and interpretable conversational design.

2. Data

The foundation of this project is a dataset of user-system interactions from the restaurant recommendation domain. The initial raw dataset consists of 25,501 utterances. During a preliminary preprocessing phase, the data was cleaned to ensure quality for model training. This involved removing 1,612 entries with "null" dialogue acts and 304 utterances marked as "unintelligible," resulting in a refined dataset of 23,889 labeled utterances. A notable characteristic of this dataset is the high frequency of repeated phrases; of the nearly 24,000 cleaned utterances, only 5,110 are unique. This high rate of duplication suggests that many users interact with the system using a limited set of common phrases, a factor that has significant implications for training machine learning models.

The language used by users in the dataset is typically informal and direct. Utterances are often short, grammatically simple, and sometimes consist of only keywords (e.g. "preferably cheap"). This conversational style reflects realistic user behavior but poses a challenge for a dialogue system. A simple rule-based approach might fail when faced with this variability, while machine learning classifiers must be robust enough to handle such noisy and abbreviated text. The prevalence of duplicate utterances, however, can be advantageous for training, as it provides the models with many examples of common interaction patterns.

The dialogue manager's logic is structured around a finite state machine, which is visualized in the state transition diagram shown in Figure 1 (see Appendix). This diagram maps out the possible paths a conversation can take, with each node representing a state in the dialogue. Before any of the training data or user inputs are processed all text is lowercased so that the algorithms do not treat the same words in different cases as different.

To illustrate the system in practice, consider the following dialogue snippet, which demonstrates how user utterances guide the system through different states.

```System: "Welcome! Let's start. What kind of restaurant are you looking for? Please inform me about your preferences (area, food, price range)."

User: "I'm looking for a Chinese restaurant in the south in the cheap price range."```

Here, the dialogue begins in the initial welcome state (State 1 in Appendix: Figure 1). The user provides a complete set of preferences in a single inform utterance. This allows the system to bypass the intermediate preference-gathering states (i.e. ask_area, ask_food) and transition directly to the suggest_restaurant state (State 5 in Appendix: Figure 1).

```System: "the missing sock is a nice place in the south part of town serving chinese food in the cheap price range."

System: "Is this suggestion okay for you?"```

Having found a match, the system offers a suggestion and, as per its logic, immediately transitions to the ask_confirmation state (State 6) to get feedback from the user. This flow demonstrates the system's ability to handle both single- and multi-intent utterances while navigating through its defined conversational states.

## 3. Machine Learning

To classify user utterances into dialogue acts, a comparative study of four different machine learning models was conducted, evaluated against two baseline systems. The entire process, from data preprocessing to evaluation, was structured to ensure robustness and reproducibility.

**Baselines and Preprocessing**

Two baseline systems were established to provide a benchmark for the machine learning classifiers. The first, a Majority Baseline, is a simple model that always predicts the most frequent dialogue act in the training data, which was found to be 'inform' (~42.5% of instances). The second, a Rule Based Baseline, employs a set of manually defined keyword-matching rules to classify utterances. For example, the presence of words like "hello" or "hi" maps to the 'hello' act. This baseline is more sophisticated but remains brittle compared to statistical models.

The preprocessing pipeline for the machine learning models was designed to convert the raw text utterances into a numerical format suitable for classification. This was achieved using a scikit-learn Pipeline object that chains a text vectorizer with a classifier. Two vectorization techniques were explored: CountVectorizer, which creates a vocabulary of known words and represents each utterance as a vector of word counts, and TfidfVectorizer, which extends this by weighting word counts based on their importance in the dataset. This vectorization step inherently handles out-of-vocabulary words by simply ignoring terms not seen during training.

Analysis of the cleaned dataset revealed a significant class imbalance, with 'inform' accounting for over 42% of the data, while acts like 'reqmore' constitute less than 0.1%. The utterance lengths are generally short, with an average of approximately 4 words, a minimum of 1, and a maximum of 23. This brevity and class imbalance are key challenges the classifiers must overcome.

**Quantitative Evaluation**

Four classifiers were trained and evaluated: Logistic Regression, Support Vector Machine (SVM), Multinomial Naive Bayes, and a Decision Tree. Each model's hyperparameters were tuned using either Optuna or GridSearchCV to find the optimal configuration. The models were trained on two versions of the dataset: the original cleaned data with duplicates, and a deduplicated version. The performance, measured by accuracy and weighted F1-score, is summarized in Table 1.

*Table 1 Performance Metrics for a variety of models on both the original and deduplicated data.*

| System | Accuracy (Original) | Accuracy (Deduplicated) |
|---|---|---|
| **Majority Baseline** | 0.4280 | 0.5841 |
| **Rule-Based Baseline** | 0.9040 | 0.8592 |
| **Logistic Regression** | 0.9869 | 0.9413 |
| **SVM** | 0.9863 | 0.9426 |
| **Multinomial Naive Bayes** | 0.9467 | 0.9048 |
| **Decision Tree** | 0.9595 | 0.9153 |

The results clearly indicate that both Logistic Regression and SVM are the top-performing models, achieving over 98% accuracy on the original dataset. The performance of all machine learning models drops on the deduplicated data, suggesting that the high number of duplicate utterances in the original dataset may have led to a degree of overfitting. Nonetheless, even on the deduplicated set, the top models significantly outperform the baselines.

**Error Analysis and Difficult Instances**

An analysis of classification errors reveals that certain dialogue acts are inherently more difficult to distinguish. A primary example is the confusion between thankyou and bye. Utterances such as "thank you goodbye" contain keywords for both acts, making classification ambiguous. The model's prediction in such cases depends heavily on the other contextual words in the utterance and the patterns learned during training.

To further probe the models' robustness, specific "difficult instances" were crafted. For example, the utterance "you are not wrong" was designed to test the handling of negation. While a human understands this as an affirmation, a simple model might incorrectly classify it based on the word "not". Another example, "phonenumer please!", tests resilience to misspellings. When evaluated on these custom sentences, the Decision Tree model, for instance, incorrectly classified both, predicting 'inform' instead of the expected 'request' and 'affirm', highlighting the limitations of the learned patterns when faced with novel or complex phrasing.

# 4. Dialog Manager

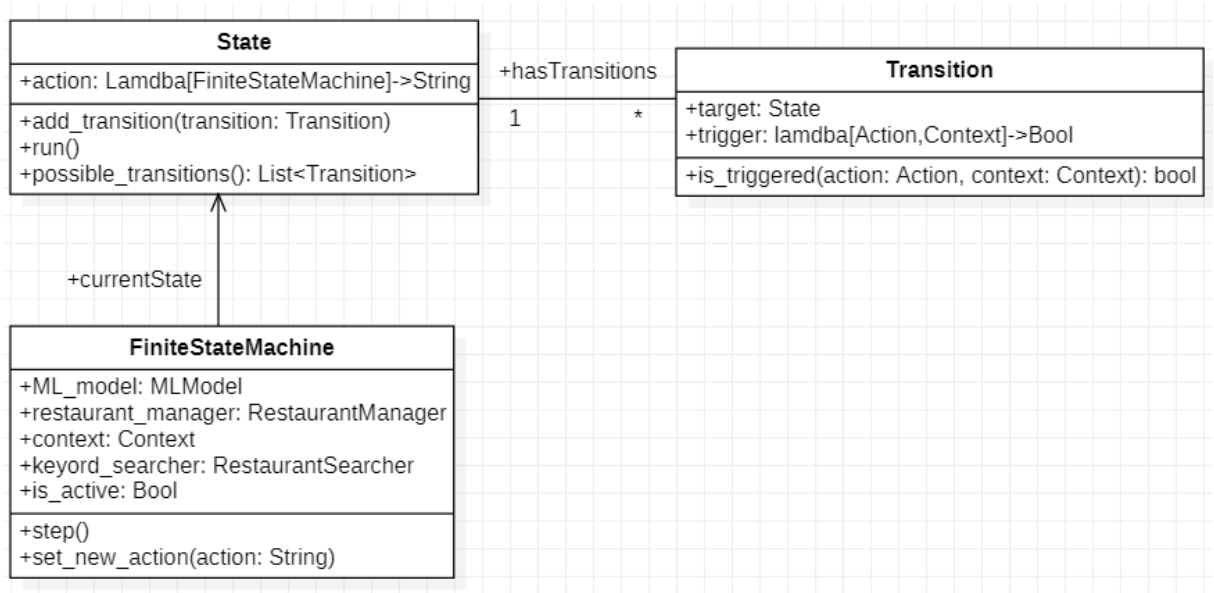


*Figure 1 presents a simplified design of the dialog manager core, based on a finite state diagram architecture.*

To ensure a clean and modular dialog manager, this project employs a Finite State Machine (FSM) design pattern. The FSM serves as the central control mechanism for managing dialog flow, with direct access to key resources such as the machine learning model and the restaurant manager, which provides information about all available restaurants.

The dialog remains active as long as the FSM's step function is running. At each step, the FSM executes the current state's action and evaluates all possible transitions. If a transition's condition is satisfied, the FSM updates the current state to the transition's target state (see Figure 1).

The FSM in this project consists of eleven distinct states. Each state will be described in the following chapters. The complete state diagram is presented in Figure 5, while a full restaurant dialog example is shown in Figure 6 and Figure 7.

## State 1: Welcome state

**System utterance template:**

- "Welcome! Let's start. What kind of restaurant are you looking for? Please inform me about your preferences (area, food, price range)."

**Transitions:**

- If area, food, and price range are known → Go to State 10 (Show possible restaurants).
- If area not known → Go to State 2 (Ask area).
- If food not known → Go to State 3 (Ask food).
- If price range not known → Go to State 4 (Ask price range).

## State 2: Ask area

**System utterance template:**

- "Which area would you like?"
- "I'm sorry, I don't recognize that area. Please choose from: {hint_options}.

**Examples:**

- "I'm sorry, I don't recognize that area. Please choose from: *north, south*."

**Transitions:**

- If all preferences known → Go to State 10 (Show possible restaurants).
- If area known but food not known → Go to State 3 (Ask food).
- If area known but price range not known → Go to State 4 (Ask price range).

## State 3: Ask food

**System utterance template:**

- "What type of food do you prefer?"
- "I'm sorry, I don't recognize that type of food. Please choose from: {hint_options}."

**Examples:**

- "I'm sorry, I don't recognize that type of food. Please choose from: *Italian, Indian*."

**Transitions:**

- If all preferences known → Go to State 10 (Show possible restaurants).
- If food known but area not known → Go to State 2 (Ask area).
- If food known but price range not known → Go to State 4 (Ask price range).

## State 4: Ask price range

**System utterance template:**

- "What price range are you looking for?"
- "I'm sorry, I don't recognize that price range. Please choose from: {hint_options}."

**Examples:**

- "I'm sorry, I don't recognize that price range. Please choose from: *moderate, expensive*."

**Transitions:**

- If all preferences known → Go to State 10 (Show possible restaurants).
- If price range known but area not known → Go to State 2 (Ask area).
- If price range known but food not known → Go to State 3 (Ask food).

## State 5: Suggest restaurant

**System utterance template:**

- " {suggestion.name} is a nice place in the {suggestion.area} part of town serving {suggestion.food} food in the {suggestion.pricerange} price range."
- "I'm sorry, there are no restaurants that match your request."

**Examples:**

- *"La Piazza* is a nice place in the *center* part of town serving *Italian* food in the *moderate price range."*

**Transitions:**

- If restaurant found → Go to State 6 (Ask confirmation).
- If no restaurant found → Go to State 8 (Ask to express preferences again).

## State 6: Ask confirmation

**System utterance template:**

- "Is this suggestion okay for you?"

**Transitions:**

- If user accepts (inform action) → Go to State 9 (Say bye).
- If user rejects (deny/negate) → Go to State 7 (Ask which part is incorrect).

## State 7: Ask which part is incorrect

**System utterance template:**

- "Which part of the suggestion was incorrect? (Area, Food, Price Range, All)"

**Transitions:**

- If all wrong → Go to State 8 (Ask to express preferences again).
- If area wrong → Go to State 2 (Ask area).
- If food wrong → Go to State 3 (Ask food).
- If price range wrong → Go to State 4 (Ask price range).

## State 8: Ask preferences again

**System utterance template:**

- "Please express your preferences again (area, food, price range)."

**Transitions:**

- If area, food, and price range are known → Go to State 10 (Show possible restaurants).
- If area not known → Go to State 2 (Ask area).
- If food not known → Go to State 3 (Ask food).
- If price range not known → Go to State 4 (Ask price range).

## State 9: Say bye

**System utterance template:**

- "Goodbye"

## State 10: Show amount of possible restaurants

**System utterance templates:**

- "I'm sorry, there are no restaurants that match your request."
- "There are {len(matches)} restaurants that match your preferences:"
- " - {r.name}: {r.food} (food), {r.pricerange} (price range), {r.area} (area), {r.food_quality} (food quality), {r.crowdedness} (crowdedness), {r.length_of_stay} (length of stay)"

**Examples:**

- "There are *2* restaurants that match your preferences:"
    - *"- La Piazza: Italian* (food), *moderate* (price range), *center* (area), *good* (food quality), *quiet* (crowdedness), *long* (length of stay)"
    - *"- Pancake: dutch* (food), *cheap* (price range), *center* (area), *good* (food quality), *busy* (crowdedness), *long* (length of stay)"
- "I'm sorry, there are no restaurants that match your request."

**Transitions:**

- If no restaurant found → Go to State 8 (Ask to express preferences again).
- If one restaurant found → Go to State 5 (Suggest restaurant)
- If more than one restaurant found → Go to State 11 (Ask extra preferences).

## State 11: Ask Extra Preferences

**System utterance template:**

- "Would you like to specify any extra preferences to narrow down the options? (touristic, assigned seats, romantic, children)"

**Transitions:**

- If no restaurant found → Go to State 8 (Ask to express preferences again).
- If one restaurant found → Go to State 5 (Suggest restaurant).

## 5. Reasoning

The reasoning component is responsible for explaining why a restaurant is, or is not, recommended based on its attributes. It operates by evaluating a fixed set of six inference rules. For each restaurant, the reasoner checks whether the antecedent conditions of a rule are satisfied by the restaurant's attributes. If a match is found, the corresponding consequence is inferred. These inferences are then translated into human-readable justifications that explain the recommendation outcome.

*Table 2 reasoning rules for dialog manager.*

| ***id*** | ***antecedent*** | ***consequent*** | ***t/f*** |
|---|---|---|---|
| **1** | cheap AND good food | touristic | True |
| **2** | romanian | touristic | False |
| **3** | busy | assigned seats | True |
| **4** | long stay | children | False |
| **5** | busy | romantic | False |
| **6** | long stay | romantic | True |

```
System: There are 2 restaurants that match your preferences:
System: - bangkok city: thai (food), expensive (price range), centre (area), poor (food quality), busy (crowdedness), short (length of stay)
System: - hk fusion: thai (food), expensive (price range), centre (area), average (food quality), moderate (crowdedness), long (length of stay)
[FSM] Current state: show_possible_restaurants, Action: inform
System: Would you like to specify any extra preferences to narrow down the options? (touristic, assigned seats, romantic, children)
You: i like a romantic place
Not recommended: bangkok city
Reasoning:
- It is busy, so it is not romantic.

Recommended: hk fusion
Reasoning:
- Guests tend to stay long, which makes it romantic.
```

Figure 2 reasoner example with romantic consequent.

```
System: - la raza: spanish (food), cheap (price range), centre (area), average (food quality), moderate (crowdedness), long (length of stay)
System: - pizza express: spanish (food), cheap (price range), centre (area), good (food quality), moderate (crowdedness), long (length of stay)
[FSM] Current state: show_possible_restaurants, Action: inform
System: Would you like to specify any extra preferences to narrow down the options? (touristic, assigned seats, romantic, children)
You: touristic
Recommended (no inferences): la raza
Reasoning:

---------------------------------------

Recommended: pizza express
Reasoning:
- It is cheap and has good food, so it attracts tourists.
```

Figure 3 reasoner example with touristic consequent.

For example, consider a scenario where a client is looking for a romantic restaurant. The reasoner examines each restaurant to determine whether its attributes satisfy the relevant rules, such as busy or long stay. In this case, Bangkok City is busy, so the reasoner infers that it is not romantic and, therefore, does not recommend it. In contrast, HK Fusion has a long stay, which satisfies the rule for romantic settings, so it is inferred to be romantic and recommended (see Figure 2).

In another scenario, the first restaurant may not trigger any reasoning rules. Since no negative inferences are made, the restaurant is still recommended because it does not contradict the

client's preferences. The second restaurant, being cheap and offering good food, is inferred to be touristic (see Table 2 and Figure 3)

```
System: Would you like to specify any extra preferences to narrow down the options? (touristic, assigned seats, romantic, children)
You: romantic
Not recommended: la raza
Reasoning:
- It is busy, so it is not romantic.

----------------------------------------

Not recommended: pizza express
Reasoning:
- Guests tend to stay long, which makes it romantic.
- It is busy, so it is not romantic.
- Contradictions found: Romantic status contradicts long stay rule.

----------------------------------------
```

Figure 4 reasoner example with contradiction romantic

However, some cases may involve contradictions. For instance, Pizza Express has a long stay, which suggests it is romantic, but it is also busy, which suggests it is not romantic. The reasoner identifies this contradiction and, to avoid client disappointment, does not recommend the restaurant (see Figure 4).

6. Configurability

We implemented several configurable features in the dialog manager, to make the dialog system adaptable for different user interaction scenarios. These options can be set interactively at runtime through the command-line interface (CLI), allowing the user to enable or disable specific functionalities before the conversation starts. The configuration options are then passed as Boolean or integer parameters to the dialog initialization function (initialize_fsm) and subsequently influence the dialog flow and system behavior.

In total, we have implemented four different configurability features: automatic speech recognition, text-to-speech output, an adjustable number of restaurant recommendations, and delayed system output to simulate a more natural interaction pace.

6.1 Feature 1: Automatic Speech Recognition (ASR)

The first configurable feature enables automatic speech recognition, allowing users to provide spoken input rather than typing. When the ASR feature is enabled (use_asr=True), the dialog system records audio from the user and converts it into text before processing it with the dialog act classifier. When the feature is disabled by the user, the system operates in standard text-input mode.

This feature is set interactively via CLI prompt:

Enable ASR (Speech-to-Text)? (y/n)

6.2 Feature 2: Text to Speech (TTS)

The second configurable feature we have implemented is the Text to Speech (TTS) feature. The TTS feature allows the system to generate spoken versions of its responses in addition to printing them on screen. When enabled (use_tts=True), each system utterance is vocalized through a text-to-speech engine. This feature enhances accessibility and realism by simulating a spoken dialog system.

This feature is set interactively via CLI prompt:

Enable TTS (Text-to-Speech)? (y/n)

6.3 Feature 3: Maximum Number of Restaurant Recommendations

The third feature of the system allows users to specify a maximum number of restaurant recommendations to display. When this feature is activated, the user is prompted to provide an integer value that sets the limit (e.g., 3 recommendations). This feature helps control output verbosity and supports testing of user satisfaction under different information loads.

This feature is set interactively via CLI prompt:

Limit number of recommendations? (y/n)
Enter maximum number of recommendations: <integer>

6.4 Feature 4: Delayed Output Response

Finally, when the fourth feature is enabled, the dialog system inserts a small artificial delay between user input and system output. This feature (use_delay=True) helps create a more natural conversational rhythm.

This feature is set interactively via CLI prompt:

> Enable a delay in the output response to have a more natural dialogue? (y/n)

6.5 Implementation Details

At runtime, the configurability options are set by the user via interactive command-line prompts defined in cli.py. For each session, the user specifies whether to enable or disable optional functionalities such as ASR, TTS, output delay, or recommendation limits. These settings are then stored in local Boolean or integer variables (use_asr, use_tts, use_delay, and restaurant_manager.max_recommendations) and passed to the finite-state machine (FSM) initializer function.

The FSM is created by calling:

```
fsm = initialize_fsm(restaurant_searcher, model, restaurant_manager, use_asr, use_tts, use_delay)
```

From there, the parameters are propagated into the dialogue management layer implemented in finite_state_machine_initializor.py. This module is responsible for constructing the dialogue state machine and controlling system behavior during execution. Each configuration flag influences the FSM logic as follows:

- **use_asr (Feature 1)**: Determines whether user utterances are captured via microphone and converted into text (when True) or typed manually (when False).
- **use_tts (Feature 2)**: Enables or disables text-to-speech output when generating system utterances.
- **use_delay (Feature 3)**: Inserts a short time.sleep() pause before printing or speaking a system response, creating more natural timing between turns in the conversation. The delay time is random between one and two seconds.
- **restaurant_manager.max_recommendations (Feature 4)**: Limits the number of restaurant results returned by the RestaurantManager object. This is enforced during the search phase in restaurant.py, where only the top *n*matches are displayed.

7. Conclusion

7.1 Discussion of the Approach for the Dialog System in General

A modular dialog-based restaurant recommendation system that combines machine learning, a finite state machine (FSM), and a reasoning component into a single, cohesive framework was implemented in this project. The system effectively walks users through the process of defining their preferences, selecting restaurants, and getting tailored suggestions. While the reasoning module adds a layer of explainability by using rule-based inferences to improve or support recommendations, the FSM structure guarantees a consistent and understandable dialog flow.

One important discovery is that a hybrid strategy, which combines rule-based logic with data-driven components, produces a reliable and interpretable system. The dialog manager exhibits flexibility and adaptability to various user environments thanks to configurable features like ASR, TTS, and random response delay. In addition to giving the discussion more realism, the configurability module lays the groundwork for upcoming multimodal extensions and usability testing.

However, several limitations remain. The current system is restricted to a fixed restaurant dataset, and the dialog flow does not dynamically adapt to unseen states or novel user intents. Furthermore, the reasoning component uses a limited set of static inference rules, which restricts its ability to generalize or handle more complex reasoning tasks. The absence of contextual memory also limits the system's ability to maintain long-term conversational coherence, especially across multiple user turns.

Potential improvements include switching from the finite-state method to a hierarchical or frame-based dialog manager and incorporating a more complex natural language understanding (NLU) module, like a transformer-based intent recognizer. This would enable more adaptable, mixed-initiative communication. Furthermore, adding probabilistic or knowledge-graph-based reasoning to the reasoning component would allow for more thorough explanations and increased precision.

7.2 Discussion of the approach for dialog act classification

The dialog act classification module forms the data-driven core of the system. The experimental methodology involved training four traditional machine learning classifiers — Logistic Regression, Support Vector Machine (SVM), Multinomial Naive Bayes, and Decision Tree — alongside two baseline systems. Preprocessing included lower-casing, tokenization, as well as an explicit deduplication step to eliminate repeated utterances. This made it possible to evaluate the models both with and without duplicated data in order to quantify the influence of redundancy on performance. However, the dataset itself was relatively small, which limited model generalization and made it more sensitive to overfitting. Despite this limitation, the results showed that the Logistic Regression and SVM models achieved the highest accuracy, around 98%, with only a minor decrease after deduplication—indicating that the classifiers were robust and not merely memorizing repeated samples.

Nevertheless, the limited number of unique utterances constrains the models' ability to generalize to unseen or paraphrased inputs. Because the classifiers rely on lexical features derived from bag-of-words and TF-IDF representations, they remain sensitive to spelling variation, word order, and synonymous expressions.

Future work could incorporate contextual embeddings from transformer-based models such as BERT or DistilBERT, which capture semantic relationships beyond surface words and can handle paraphrases more effectively. Another possible improvement would be a hierarchical or joint classification framework where intent recognition and dialogue-act prediction are learned together. Combining these advancements with real-world restaurant data sources would make the system more realistic and allow it to work with up-to-date restaurant information, helping turn the prototype into a practical conversational agent.

7.3 Final Remarks

All things considered, the project shows that a modular hybrid method that combines deterministic state management, statistical classification, and logical reasoning could yield a dialog system that is reliable, interpretable, and extendable. Although the system's contextual comprehension and reasoning capabilities are limited, it offers a solid basis for further research into adaptive, multimodal conversational agents.

8. Contributions

Team Member	Tasks: *Since everyone took a different amount of time to complete the assignment, we did not include the hours.*
Bram	• Logistic Regression Classifier • State Diagram • States & Transitions • Updated ReadMe for Part 1b • Working on 2 Extra Features • Report: Configurability + Conclusion + Final check
Lenny	• Naive Multinomial Bayes Classifier • States & Transitions • Feedback for States & Transitions (Redesigning the Diagram) • Adding zip code, phone number • Report: Title + Introduction + Abstract
Shady	• Data Preprocessing • SVM Classifier • CLI (1A/1B/1C) • Feedback 1A (Refactoring) • Restaurant Lookup Function • State Debugging & Edge Cases • Feedback 1B (Refactoring) • ASR & TTS • Report: Data + Machine Learning
Dirk-Jan	• Baseline Systems • Decision Tree Classifier • Evaluation • State Diagram • States & Transitions • Keyword Matching • Reasoner • Report: Dialog Manager + Reasoning

9. Appendix

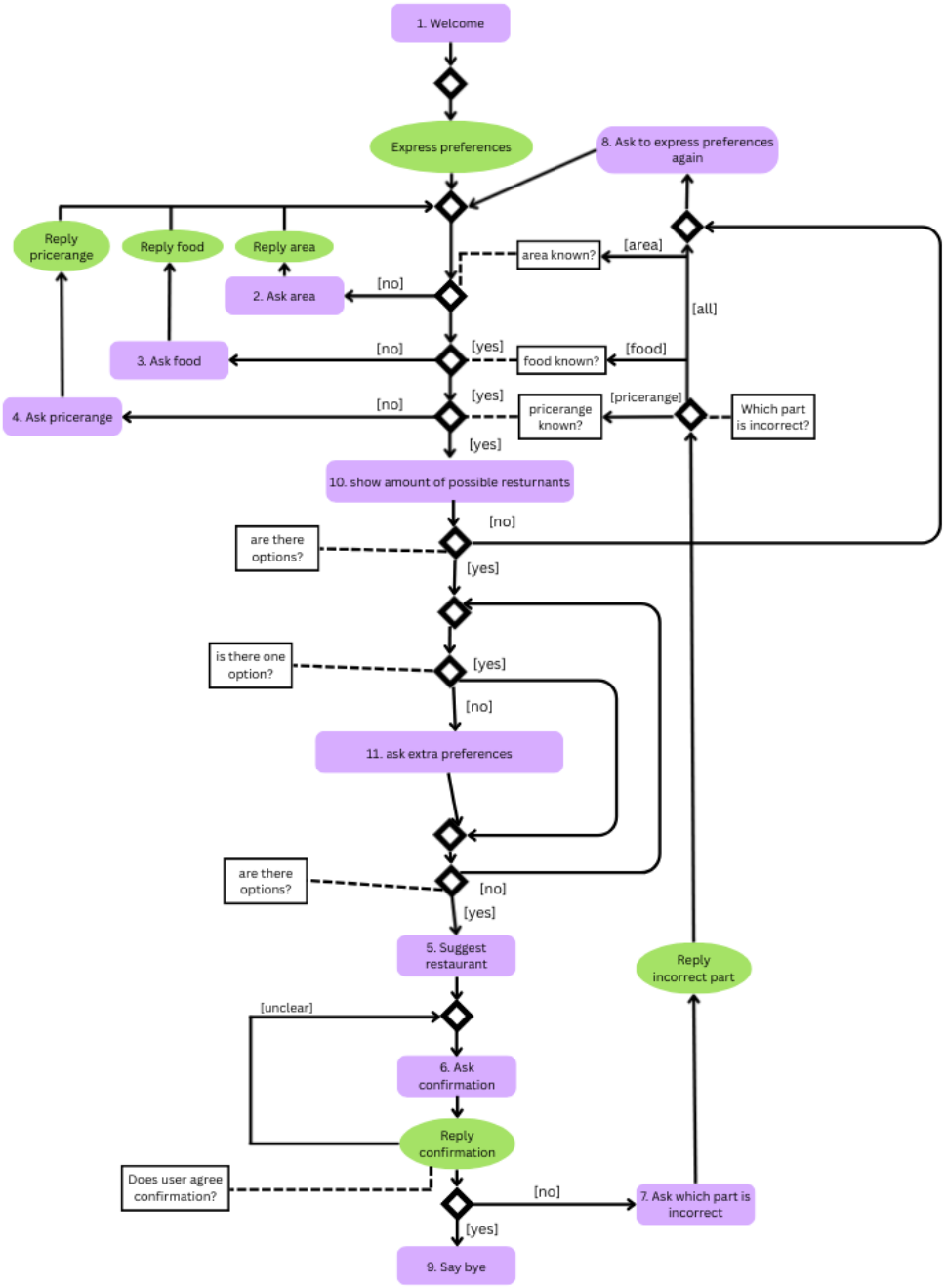

Figure 5 state diagram of restaurant dialog manager.

```
--------------------------------------------------------------------------------------------------
                                Welcome to the Restaurant Dialogue System!
--------------------------------------------------------------------------------------------------
System: Welcome! Let's start. What kind of restaurant are you looking for? Please inform me about your preferences (area, food, price range).
You: i would like some thai expensive food
[FSM] Current state: welcome, Action: inform
System: Which area would you like?
You: centre
[FSM] Current state: ask_area, Action: inform
System: There are 2 restaurants that match your preferences:
System: - bangkok city: thai (food), expensive (price range), centre (area), poor (food quality), busy (crowdedness), short (length of stay)
System: - hk fusion: thai (food), expensive (price range), centre (area), average (food quality), moderate (crowdedness), long (length of stay)
[FSM] Current state: show_possible_restaurants, Action: inform
System: Would you like to specify any extra preferences to narrow down the options? (touristic, assigned seats, romantic, children)
You: i like a romantic place
Not recommended: bangkok city
Reasoning:
- It is busy, so it is not romantic.

----------------------------------------

Recommended: hk fusion
Reasoning:
- Guests tend to stay long, which makes it romantic.

----------------------------------------

[FSM] Current state: ask_extra_preference, Action: inform
1 restaurants match the preferences.
System: hk fusion is a nice place in the centre part of town serving thai food in the expensive price range.
[FSM] Current state: suggest_restaurant, Action: inform
System: Is this suggestion okay for you?
You: no nevermind i want something else
[FSM] Current state: ask_conformation, Action: negate
System: Which part of the suggestion was incorrect? (Area, Food, Price Range, All)
You: All
[FSM] Current state: ask_part_incorrect, Action: thankyou
System: Please express your preferences again (area, food, price range).
You: i want a cheap spanisch restaurant in the centre area
[FSM] Current state: ask_to_express_preference, Action: inform
System: There are 2 restaurants that match your preferences:
System: - la raza: spanish (food), cheap (price range), centre (area), average (food quality), moderate (crowdedness), long (length of stay)
System: - pizza express: spanish (food), cheap (price range), centre (area), good (food quality), moderate (crowdedness), long (length of stay)
[FSM] Current state: show_possible_restaurants, Action: inform
```

Figure 6 full restaurant dialog example part 1.

```
System: Would you like to specify any extra preferences to narrow down the options? (touristic, assigned seats, romantic, children)
You: touristic
Recommended (no inferences): la raza
Reasoning:

----------------------------------------

Recommended: pizza express
Reasoning:
- It is cheap and has good food, so it attracts tourists.

----------------------------------------

[FSM] Current state: ask_extra_preference, Action: inform
2 restaurants match the preferences.
System: la raza is a nice place in the centre part of town serving spanish food in the cheap price range.
[FSM] Current state: suggest_restaurant, Action: inform
System: Is this suggestion okay for you?
You: yes
[FSM] Current state: ask_conformation, Action: affirm
System: Goodbye!
[FSM] Current state: bye, Action: bye
```

Figure 7 full restaurant dialog example part 2